

Padrões de Projeto (Factory Method)

QXD0068 - Reuso de Software

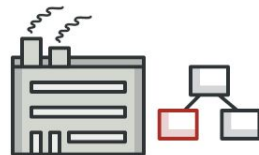
Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br



Agenda

- Introdução
- Padrão Factory
- Problema
- Solução
- Analogia do Mundo Real
- Estrutura do Padrão
- Exemplo em Kotlin e Python
- Aplicabilidade
- Prós e Contras
- Implementação



Factory Method

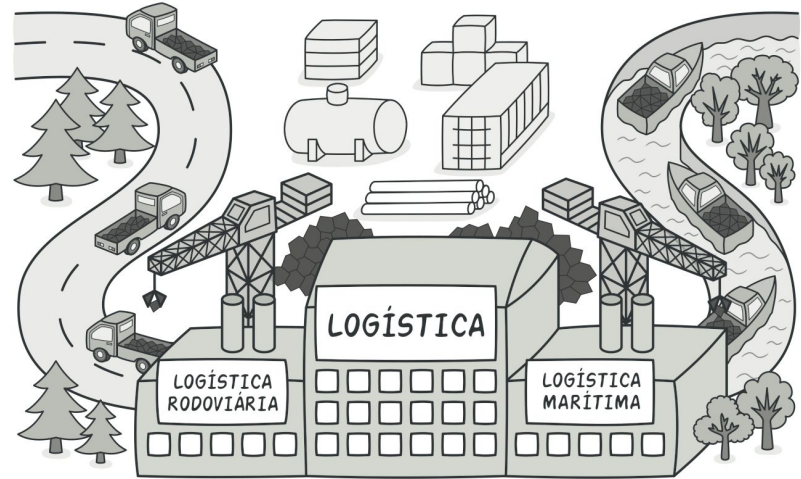
Fornece uma interface para criar objetos em uma superclasse, mas permite que as subclasses alterem o tipo de objetos que serão criados.

Introdução

- Padrão criacional que define uma interface para criar objetos em uma superclasse, permitindo que subclasses decidam qual tipo de objeto instanciar.
- Cria objetos sem acoplar o código às classes concretas.

Padrão Factory

- O Factory Method é um padrão criacional de projeto que fornece uma interface para criar objetos em uma superclasse, mas permite que as subclasses alterem o tipo de objetos que serão criados.

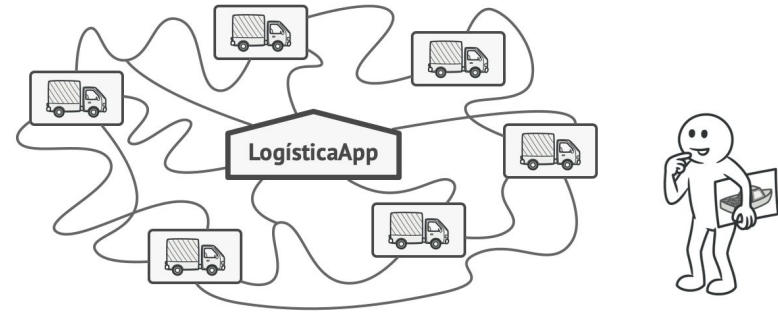


Problema

- Imagine que você está criando uma aplicação de gerenciamento de logística. A primeira versão da sua aplicação pode lidar apenas com o transporte de caminhões, portanto a maior parte do seu código fica dentro da classe Caminhão. Depois de um tempo, sua aplicação se torna bastante popular. Todos os dias você recebe dezenas de solicitações de empresas de transporte marítimo para incorporar a logística marítima na aplicação.
- A adição de uma nova classe de transporte ao programa causa um problema. Adicionar uma nova classe ao programa não é tão simples se o restante do código já estiver acoplado às classes existentes.
- Boa notícia, certo? Mas e o código? Atualmente, a maior parte do seu código é acoplada à classe Caminhão. Adicionar Navio à aplicação exigiria alterações em toda a base de código. Além disso, se mais tarde você decidir adicionar outro tipo de transporte à aplicação, provavelmente precisará fazer todas essas alterações novamente. Como resultado, você terá um código bastante sujo, repleto de condicionais que alteram o comportamento da aplicação, dependendo da classe de objetos de transporte.

Problema

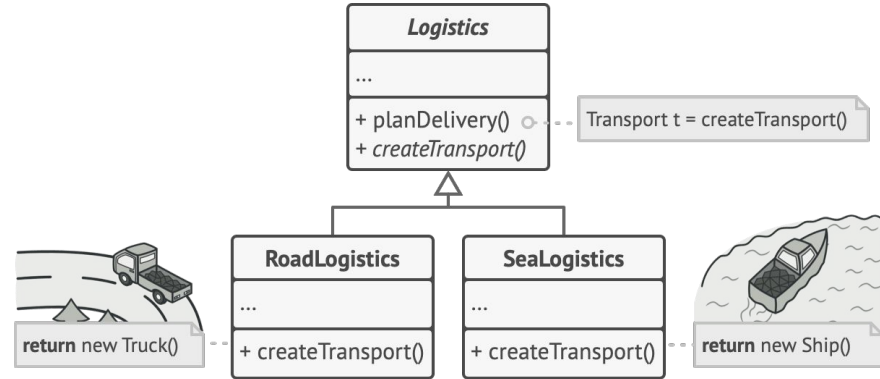
- Uma aplicação de logística só cria caminhões.
- Ao precisar incluir navios, o código torna-se cheio de condicionais e difícil de manter.
- Cada novo tipo de transporte exige alterar o código existente → alto acoplamento.



Adicionar uma nova classe ao programa não é tão simples se o restante do código já estiver acoplado às classes existentes.

Solução

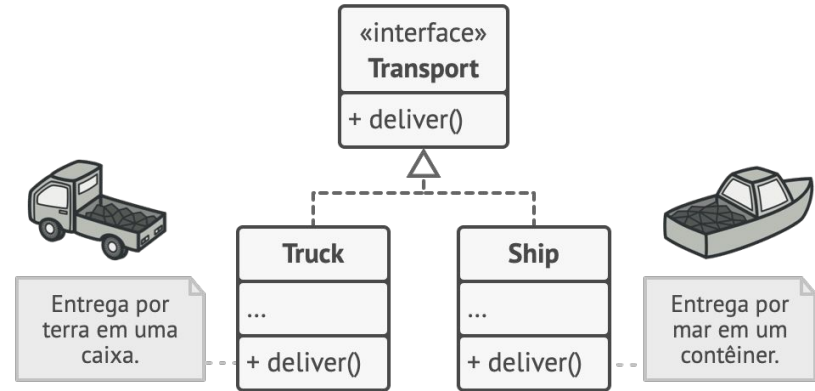
- Substituir chamadas diretas ao `new` por um método fábrica, que retorna um produto abstrato.
- As subclasses sobreescrevem o método fábrica para decidir qual objeto concreto criar.
- Assim, o código cliente continua funcionando com a interface comum (Transporte).



As subclasses podem alterar a classe de objetos retornados pelo método fábrica.

Analogia do Mundo Real

- Uma empresa de logística tem filiais diferentes:
 - A Logística Terrestre cria caminhões.
 - A Logística Marítima cria navios.
 - Cada filial sabe como fabricar o tipo de transporte que usa, mas todas seguem a mesma política de entrega.



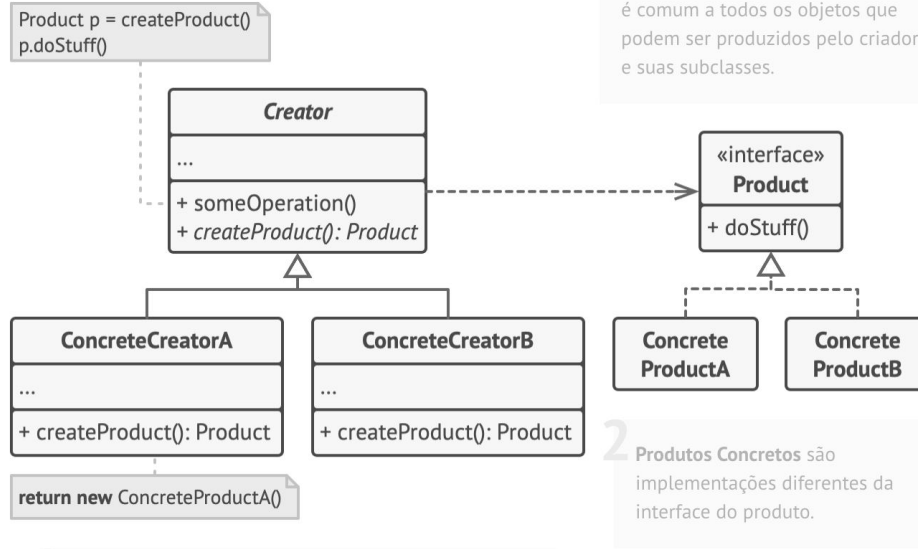
Estrutura do Padrão

- Produto → Interface comum para todos os objetos criados.
- Produto Concreto → Implementações específicas (ex.: Caminhão, Navio).
- Criador → Define o método fábrica.
- Criador Concreto → Sobrescreve o método fábrica para criar produtos específicos.

3 A classe **Criador** declara o método fábrica que retorna novos objetos produto. É importante que o tipo de retorno desse método corresponda à interface do produto.

Você pode declarar o método fábrica como abstrato para forçar todas as subclasses a implementar suas próprias versões do método. Como alternativa, o método fábrica base pode retornar algum tipo de produto padrão.

Observe que, apesar do nome, a criação de produtos **não** é a principal responsabilidade do criador. Normalmente, a classe criadora já possui alguma lógica de negócio relacionada aos produtos. O método fábrica ajuda a dissociar essa lógica das classes concretas de produtos. Aqui está uma analogia: uma grande empresa de desenvolvimento de software pode ter um departamento de treinamento para programadores. No entanto, a principal função da empresa como um todo ainda é escrever código, não produzir programadores.



1 O **Produto** declara a interface, que é comum a todos os objetos que podem ser produzidos pelo criador e suas subclasses.

2 **Produtos Concretos** são implementações diferentes da interface do produto.

4 **Criadores Concretos** sobrescrevem o método fábrica base para retornar um tipo diferente de produto.

Observe que o método fábrica não precisa **criar** novas instâncias o tempo todo. Ele também pode retornar objetos existentes de um cache, um conjunto de objetos, ou outra fonte.

```
interface Transporte { fun entregar() }

class Caminhao : Transporte { override fun entregar() = println("Entrega por terra") }
class Navio : Transporte { override fun entregar() = println("Entrega por mar") }

abstract class Logistica {
    fun planejarEntrega() { val t = criarTransporte(); t.entregar() }
    abstract fun criarTransporte(): Transporte
}

class LogisticaTerrestre : Logistica() { override fun criarTransporte() = Caminhao() }
class LogisticaMaritima : Logistica() { override fun criarTransporte() = Navio() }
```

Exemplo



```
from abc import ABC, abstractmethod
```

```
class Transporte(ABC):
```

```
    @abstractmethod
```

```
    def entregar(self): pass
```

```
class Caminhao(Transporte):
```

```
    def entregar(self): print("Entrega por terra")
```

```
class Navio(Transporte):
```

```
    def entregar(self): print("Entrega por mar")
```

```
class Logistica(ABC):
```

```
    def planejar_entrega(self):
```

```
        t = self.criar_transporte()
```

```
        t.entregar()
```

```
    @abstractmethod
```

```
    def criar_transporte(self): pass
```

```
class LogisticaTerrestre(Logistica):
```

```
    def criar_transporte(self): return Caminhao()
```

```
class LogisticaMaritima(Logistica):
```

```
    def criar_transporte(self): return Navio()
```

Exemplo



Aplicabilidade

- **Use Factory quando:**
 - O código não deve depender de classes concretas.
 - Você quer permitir extensões sem alterar o código existente.
 - Deseja centralizar a lógica de criação em um único ponto.

Prós e Contras

- **Vantagens:**
 - Desacopla a criação do uso dos objetos
 - Facilita extensão e manutenção (OCP e SRP)
- **Desvantagens:**
 - Aumenta o número de classes
 - Exige hierarquia bem planejada

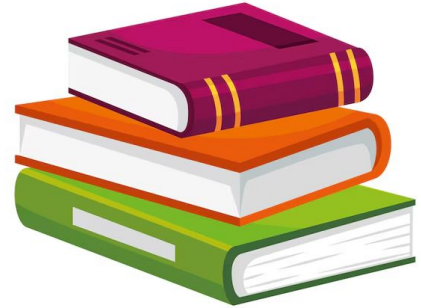
Bibliografia Básica

- MILI, Hafedh. Reuse-based software engineering: techniques, organization and measurement. New York: Wiley, 2002. 636 p. ISBN 0471398195.
- EZRAN, M.; MORISIO, M.; TULLY, C. Practical software reuse. Berlim: Springer, 2002
- POHL, K.; BÖCKLE, G.; LINDEN, F. J. Software product line engineering: foundations, principles and techniques. Berlim: Springer, c2010. xxvi, 467 p. ISBN 9783642063640 (broch.).



Bibliografia Complementar

- SOMMERVILLE, I. Engenharia de software. 7. ed. São Paulo: Pearson Addison-Wesley, 2007.
- PRESSMAN, R. Engenharia de software. 6. ed. São Paulo: Mc Graw-Hill, 2006. ISBN 8586804576
- GAMMA, E.; HELM, JOHNSON, R.; R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000. 364 p.
- BRAUDE, Eric J.; BERNSTEIN, Michael E. Software engineering: modern approaches. 2nd ed. Hoboken, New Jersey: J. Wiley & Sons, 2011. xvi, 782 p.
- BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. Software architecture in practice. 3rd ed. Upper Saddle River, NJ: Addison-Wesley, c2013. xix, 589 p. (SEI series in software engineering). ISBN 9780321815736 (enc.).



Obrigado! Dúvidas?



Universidade Federal do Ceará - *Campus* Quixadá

Prof. Francisco Victor da Silva Pinheiro
victorpinheiro@ufc.br

